

System Analysis Document

Introduction

This system analysis document introduces the strategic and technical foundation for Yama-inventory. This system is designed to solve manual, disjointed supply chain management issues into an automatic flow where the AI can communicate with other departments via emails to make restocking decisions based on any scenario based on the company's preferences.

- **Architecture:** A client-server architecture with a React-based frontend dashboard, a FastAPI backend serving REST and WebSocket endpoints, and an autonomous AI orchestration loop.
- **Data Flows:** How supply chain events (e.g., unread supplier emails, sales orders) flow through the backend, are enriched by internal data repositories, analyzed by the ILMU AI model before making decisions which are displayed to the user via WebSockets.
- **Model Process:** The end-to-end workflow of the agentic loop, which involves reasoning over tool schemas, requesting data from internal departments, checking for cross-SKU conflicts, and outputting structured JSON decisions (PO/RFQ drafts).
- **Role of Reference:** This document serves as a high-level reference guide for developers, QA testers, and stakeholders involved in maintaining and scaling the platform.

Background

The company operates a complex supply chain involving multiple raw materials (e.g., Raw Steel Component X, Aluminum Casting Y) and finished SKUs. Previously, the coordination between sales orders, inventory levels, financial budgets, and supplier communications was manual and siloed, leading to stockouts, cross-SKU conflicts, and delayed responses to critical supplier alerts (e.g., shipment delays or price hikes).

Previous Version

The existing processes relied on isolated CSV/spreadsheet tracking and manual email monitoring. The "previous version" of the workflow involved a human operator manually cross-referencing sales orders with inventory logs, reading supplier emails one-by-one, and manually drafting Purchase Orders (POs) and Request for Quotes (RFQs). The new capabilities introduced in Z.AI feature a centralized dashboard, continuous background email polling, and an AI agent that automatically detects issues and performs cross-department data aggregation.

Target Stakeholders

Stakeholders	Roles	Expectations
Supply Chain Manager / Procurement	Reviews AI-generated inventory decisions, monitor workflow alerts to solve, ensuring production workflow is smooth.	A unified dashboard displaying all actions taken by the AI, as well as their reasoning as well as setting preferences for AI to follow

Sales Team	Generates sales orders that dictate upcoming demand for finished goods.	Assurance that raw materials will be available to fulfill their sales orders on time without cross-SKU conflicts. Will send follow up email if lacking information.
Suppliers (Vendors)	Receive Request for Quotes (RFQs) and Purchase Orders (POs) generated by the system.	Timely, clear, and standardized communication regarding new orders or inquiries.
Development Team	Builds and maintains the Z.AI platform and its micro-components.	Clear API usage contracts, modular architecture (e.g., Dependency Inversion for the LLM Client), and well-documented agentic flows.
QA Team	Validates system behavior, API endpoints, and AI reasoning boundaries.	Defined mock scenarios, clear JSON schema constraints for the AI, and test coverage for the background polling loop.

System Architecture & Design

High Level Architecture

Type	Details
System	Web Dashboard (React, Vite)
Architecture	Client-Server API with Agentic Background Loop

Z.AI is structured as a client-server-based system. The client is a React Web Dashboard used by Supply Chain Managers. The dashboard communicates with a FastAPI Backend via RESTful APIs (for current status/data) and WebSockets (for real-time AI reasoning streams). The backend architecture consists of:

1. **Data Repository:** Currently utilizing CSV files to simulate department databases (Inventory, Sales, Finance, Logistics, Manufacturing).
2. **Email Poller:** An asynchronous background loop that polls an IMAP mailbox every 10 minutes for unread supplier emails.
3. **Agentic Orchestrator:** The core intelligence that uses the `ILMUClient` to analyze input invoke tools (e.g., `get_sales_data`), generate candidate decisions and execute the most optimal decision``.

LLM as Service Layer

The ILMU LLM is integrated as the core reasoning engine, heavily leveraging OpenAI SDK compatibility.

Dependency Diagram & Flow:

- **Prompt Construction:** The `Orchestrator` constructs a system prompt demanding JSON adherence and checks for cross-SKU conflicts. It passes user input and a suite of `ToolRegistry` schemas to the ILMU API.
- **Context Window Management:** To enforce token limits and prevent context overflow, email bodies are explicitly truncated before being passed to the AI (`body[:2000]`). The agentic loop is strictly capped at `MAX_STEPS = 10` to prevent runaway API usage.
- **Response Parsing:** The backend receives the `ChatCompletionMessage`. If `.tool_calls` are present, the backend automatically maps the function names to the `ToolRegistry`, executes the local python functions (e.g., `get_inventory_data()`), appends the JSON results to the message history, and re-prompts the LLM.
- **Final Delivery:** When no tool calls are requested, the final string is cleaned of markdown fences (e.g., ````json`), parsed as a JSON object, and streamed to the frontend via WebSocket.

Sequence Diagram (Place Order / AI Recommendation Flow)

1. **Trigger:** The codebase is triggered by a new email -> Regularly monitor email inbox every 10 mins
2. **WebSocket Init:** Frontend establishes a WebSocket connection (`/ws/orchestrator`) and sends the payload.
3. **Agent Step 1:** Backend sends the prompt and tool schemas to ILMU API.
4. **Tool Invocation:** ILMU API returns a request to call `get_inventory_data` and `get_sales_data`.
5. **Local Execution:** Backend executes these tools against the Data Repository and returns the tabular data to ILMU.
6. **Final Decision:** ILMU analyzes the cross-SKU conflicts, decides to order 500 units of RAW-001, and sends the email to the supplier.
7. **Delivery:** Backend parses the JSON and pushes the `result` event to the frontend, displaying the Draft PO and decision taken. -> display action status

Technological Stack

- **Frontend:** React 19, Vite, Recharts, React Router — Web-based dashboard for data visualization and AI chat interaction.
 - **Backend:** FastAPI, Uvicorn, asyncio, pandas — RESTful endpoints and WebSocket server handling the agentic loop and background tasks.
 - **AI Engine:** ILMU API (YTL AI Labs) — Handling prompt inference, multi-step tool calling, and text-to-JSON structuring. Accessed via the `openai` Python SDK.
 - **Database / Storage:** Local CSV Datasets (Simulating internal PostgreSQL/ERP databases) and in-memory lists for active email alerts.
 - **External Integrations:** IMAP Server (for ingesting supply chain emails).
-

Key Data Flows

Data Flow Diagram (DFD) Narrative:

- 1. **Email Ingestion:** External emails arrive at the Inbox. The `email_reader.py` background task fetches UNSEEN emails via IMAP.
- 2. **Email AI Classification:** The email body (truncated) is sent to ILMU (`ilmu-glm-5.1` model) to determine `is_issue` (e.g., shipment delay). If true, it is stored in the in-memory alert list.
- 3. **Data Repository:** Core department data (Inventory, Sales, Finance, Logistics) is read from CSV files using `pandas`.
- 4. **Orchestrator Loop:** Data moves from the CSVs -> Python Tools -> ILMU API -> JSON Response -> WebSocket -> React Dashboard state.

Functional Requirements & Scope

Minimum Viable Product (MVP)

#	Features	Description
1	Action command centre	Users can view past and current action taken by AI and the explanation for the decision taken
2	Automated information handling	Background service fetches unread supplier emails, classifies them using AI, and automatically follows up with sender for missing data
3	Agentic Decision Engine	Users can trigger the Z.AI agent to reason over department data, identify cross-SKU resource conflicts, and make quantifiable reorder decisions. + company preferences and guardrails
4	Automated Draft Generation	Upon reaching a decision, the system automatically generates draft Purchase Orders, Work Orders, and Supplier RFQ Emails and sends them to corresponding departments.

Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	The backend must handle concurrent WebSocket connections from multiple Supply Chain Managers.	Use of <code>asyncio</code> in FastAPI endpoints and threading (<code>asyncio.to_thread()</code>) for blocking IMAP network calls.

Reliability	The agent must not fall into an infinite loop of API calls if it gets confused by data.	Strict enforcement of <code>MAX_STEPS = 10</code> in the <code>Orchestrator</code> class. Graceful fallback to Mock reasoning if the API key is missing.
Maintainability	The codebase must allow easy swapping of LLM providers or addition of new tools.	Implemented using Dependency Inversion. The orchestrator depends on the <code>LLMClient</code> interface and a <code>ToolRegistry</code> abstraction.
Token Latency	LLM responses should not block the user interface entirely while "thinking".	WebSockets stream the internal "Reasoning / Calling Tool..." status steps to the UI so the user knows progress is happening.
Cost Efficiency	Prevent massive token expenditures on long, irrelevant promotional supplier emails.	Input text bodies are strictly truncated <code>body[:2000]</code> before classification, and a fast model (<code>ilmu-glm-5.1</code>) is used for email triage while the main orchestrator uses (<code>ilmu-1.0</code>).

Out of Scope / Future Enhancements

- **Direct ERP Database Integration:** Replacing the mock CSV `DataRepository` with live connectors to SAP/Oracle databases via robust ETL pipelines. It could also access reports on arriving supplies and production progress to update the warehouse database.
- **Predictive Demand Forecasting:** Expanding the AI to run time-series forecasting (e.g., using ARIMA or specialized ML models) on historical sales data to predict future seasonality, rather than just reacting to current pending orders.

Monitor, Evaluation, Assumptions & Dependencies

Technical Evaluation:

- **Grayscale Rollout & A/B Testing:** The system utilizes a modular tool registry. New tools (e.g., `get_predictive_forecast`) can be introduced for 5% of reasoning flows to evaluate if they improve the accuracy of the PO recommendations before full rollout.
- **Emergency Rollback (ER) & Golden Release:** If the ILMU API experiences severe degradation, the system is designed to gracefully fallback to either rule-based keyword detection (for emails) or a Mock reasoning flow (for the orchestrator), ensuring the dashboard does not completely crash.

Monitoring & Priorities:

- **Priority Matrix:**
 - **P1 Critical:** Complete failure of the IMAP poller or ILMU API authentication. Action: System throws visual error flags; fallback logic engages; alert devops.
 - **P2 High:** JSON parsing failures from the LLM. Action: The system strips markdown fences. If it still fails, the error is streamed to the user and the agent step halts safely.

Assumptions:

- Supply Chain Managers have access to a modern web browser to view the React dashboard.
- The email inbox (IMAP) monitored by the system is dedicated to supply chain/supplier communications, reducing the noise-to-signal ratio.
- The provided `.env` credentials for IMAP and ILMU API have sufficient permissions and rate limits for continuous operation.
- Production is always happening at a constant rate. Can always be done by deadline unless stated otherwise in email.
- All reports on sales, productions and any work related data are sent via email

External Dependencies:

Tools	Purpose	Risks & Mitigation
ILMU API (Z.AI)	The core LLM engine handling prompt inference, tool calling, and generating structured JSON output.	High Risk: Rate limits or API downtime. <i>Mitigation:</i> Fallback to <code>run_mock()</code> and <code>keyword_fallback()</code> to ensure graceful degradation.
OpenAI Python SDK	Used as the communication client to interact with the ILMU API seamlessly.	Low Risk: Highly stable SDK.
IMAP Server	Used to fetch incoming emails from suppliers regarding delays/price changes.	Medium Risk: Connection timeouts. <i>Mitigation:</i> Wrapped in <code>try/except</code> and executed in a background thread to prevent blocking the main API thread.

Project Management & Team Contributions

The timeline and roles represent the hackathon lifecycle (Approx. 10 days).

- **Days 1-2 (Planning & Architecture):** Define the agentic loop, identify the cross-SKU conflict use case, structure the FastAPI + React template.
- **Days 3-5 (Backend & AI Logic):** Implement the Dependency Inverted `LLMClient`, the `ToolRegistry`, and the mock CSV `DataRepository`. Prompt engineering for strict JSON output.
- **Days 6-8 (Frontend & Real-time Integration):** Build the React dashboard, hook up WebSocket streaming (`/ws/orchestrator`), and visualize the tables (Inventory, Sales).
- **Days 9-10 (Refinement & Polling):** Add the asynchronous IMAP email polling task, refine fallback behaviors, and finalize documentation.

Recommendations for Future Scalability:

- Transition from local CSV data parsing (which loads the file on every API call) to a caching layer (like Redis) or an actual indexed Relational Database (PostgreSQL) to handle much larger manufacturing datasets efficiently.

